

Project Overview : Banana Hunter is a JavaFX game where the player controls a monkey to collect bananas while avoiding banana peel traps within a limited time. The game contains multiple levels with increasing difficulty. The player earns points by collecting coins, and the game ends when the player completes all levels, hits a trap, or runs out of time. The project demonstrates software engineering concepts such as OOP, event-driven programming, modular design, and file serialization.

System Architecture (MVC) : The game follows the Model–View–Controller (MVC) architecture for better organization and maintainability.

The Model contains the game data and logic:

- **Player** : player position and movement
- **Coin** : collectible items
- **Trap** : obstacles
- **Controller** : manages game rules, score, levels, collisions, timer, and win/lose conditions

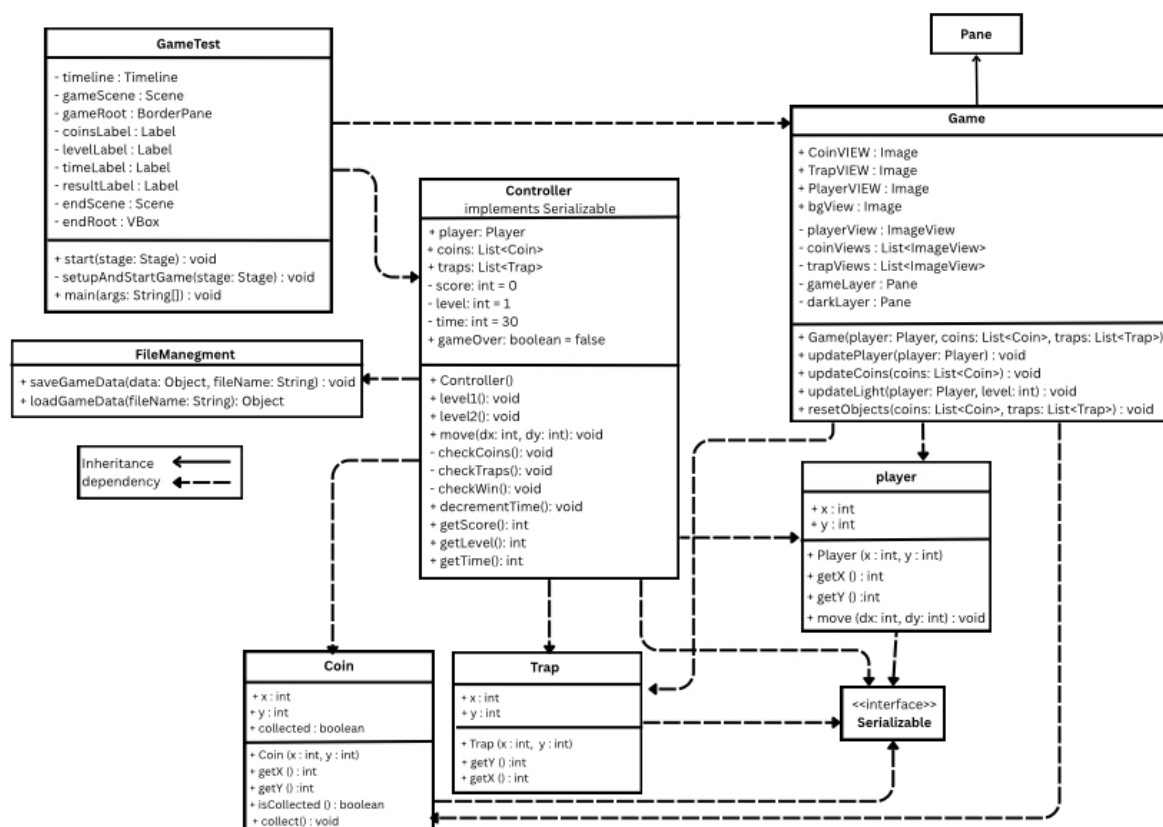
View The View handles the graphical interface using JavaFX:

- **Game** : displays the player, coins, traps, and background
- **GameTest** : main application that manages scenes and UI elements such as score, level, and timer

Controller Layer (Input & Flow) : The Controller manages:

- Keyboard input
- JavaFX Timeline game loop
- Updating game state and refreshing the view
- Scene transitions and level progression

UML Class Diagram



Key Classes and Components

GameTest

- Entry point of the application
- Manages game states and UI screens
- Handles user input and game loop execution

Game

- Responsible for rendering all visual elements
- Updates positions of game objects
- Reflects changes from the Controller

Controller

- Core logic of the game
- Manages movement, scoring, levels, and time
- Detects collisions and determines game state

Player

- Represents the player character
- Stores position and movement behavior

Coin

- Represents collectible items
- Tracks whether the coin has been collected

Trap : Represents obstacles that end the game on collision

FileManegment : Handles saving and loading game state using serialization

Persistence : The game uses Java serialization to save and load progress.

Methods used: `oos.writeObject(data);` , Load: `ois.readObject();`

Saved Data Includes:

- Player position
- Score
- Level
- Coins and traps state
- Game progress

Design Patterns : The project uses the MVC design pattern:

- **Model** : game data and logic
- **View** : graphical interface

- **Controller** : user input and game flow

MVC improves organization, readability, and maintainability.

Advanced Programming Features : The project demonstrates:

- Object-Oriented Programming (OOP)
- Event-driven programming
- JavaFX GUI development
- File serialization
- Lambda expressions
- Modular software design

Testing : The following features were tested successfully:

- Player movement
- Coin collection
- Trap collision detection
- Level progression
- Timer system
- Save/load functionality
- Scene transitions

Screenshot

